Cyprus University of Technology

CSE 226
Algorithms and Complexity

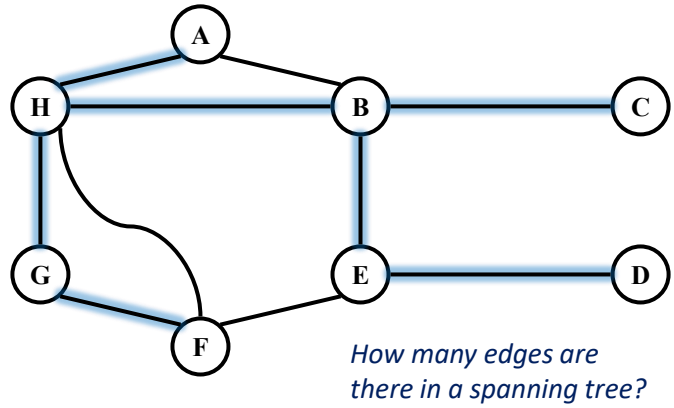
Lecture 12
Minimum Spanning Trees (Part 1)
Ελαφρύτατα Συνδετικά Δένδρα (Μέρος 1)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

Spanning Tree

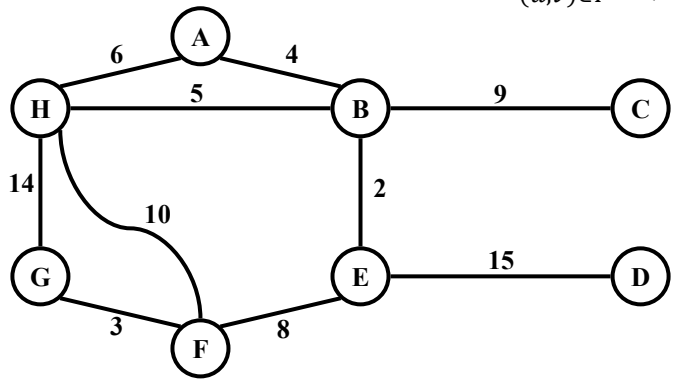
- A **spanning tree** (**συνδετικό δέντρο**) T of an undirected graph G is a subgraph that includes all the vertices of G that is a tree



How many edges are there in a spanning tree? $V - 1$

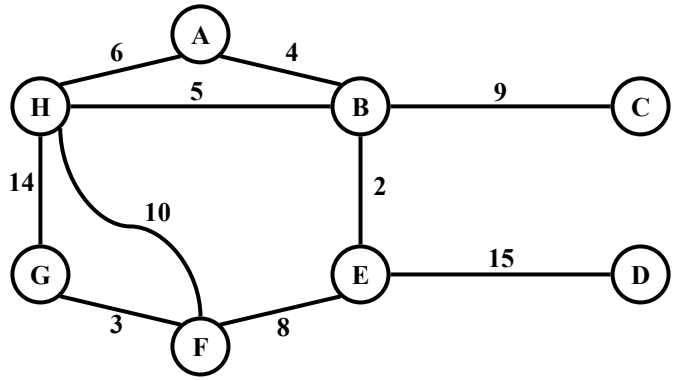
Minimum Spanning Tree

- **Problem:** given a connected, undirected, weighted graph, find a **spanning tree** (**συνδετικό δέντρο**) T using edges that minimize the total weight, $w(T) = \sum_{(u,v)\in T} w(u,v)$



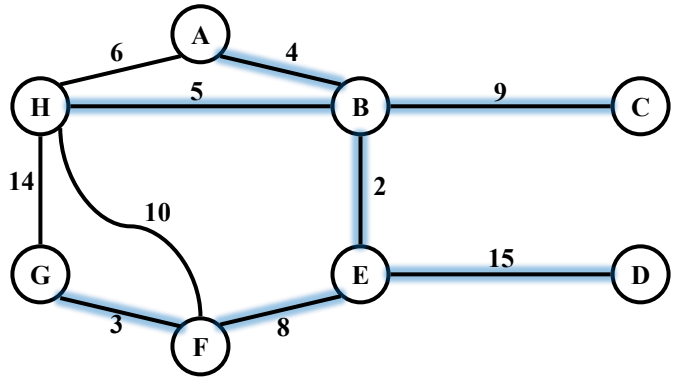
Minimum Spanning Tree

- Which edges form the **Minimum Spanning Tree** (**MST**) (**Ελαφρύτατο Συνδετικό Δένδρο**) of the below graph?



Minimum Spanning Tree

- Answer:

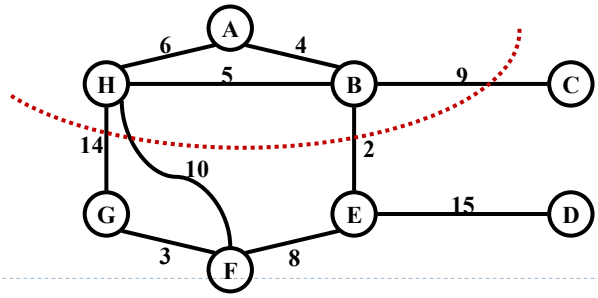


Minimum Spanning Tree

- MSTs satisfy the **optimal substructure** (**βέλτιστη υποδομή**) property: an optimal tree is composed of optimal subtrees
  - Let T be an MST of G with an edge (u, v) in the middle
  - Removing (u, v) partitions T into two trees T_1 and T_2
  - **Claim:** T_1 is an MST of $G_1 = (V_1, E_1)$, and T_2 is an MST of $G_2 = (V_2, E_2)$ (*Do V_1 and V_2 share vertices? Why?*)
  - **Proof:** $w(T) = w(u, v) + w(T_1) + w(T_2)$
    There can't be a better tree than T_1 or T_2, or T would be suboptimal

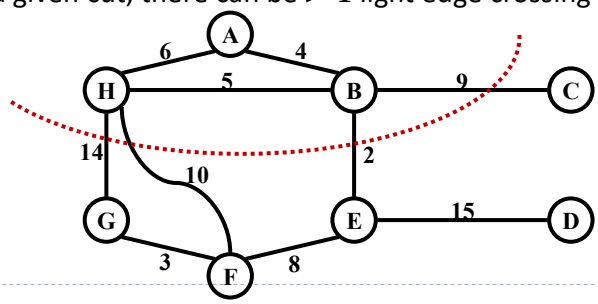
Useful Definitions

- Let $S \subset V$ and $A \subseteq E$
- A **cut** (**τομή**) $(S, V - S)$ is a *partition* (*διαμέριση*) of vertices into disjoint sets S and $V - S$
- Edge $(u, v) \in E$ **crosses** (**διασχίζει**) cut $(S, V - S)$ if one endpoint is in S and the other is in $V - S$



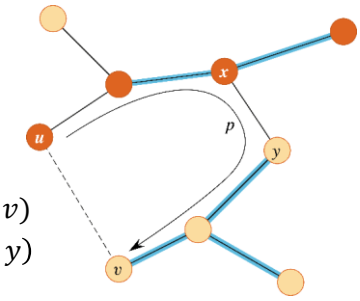
Useful Definitions

- A cut **respects** (**σέβεται**) a set of edges A if no edge in A crosses the cut
- An edge is a **light edge** (**ελαφρά ακμή**) crossing a cut if and only if its weight is minimum over all edges crossing the cut
  - For a given cut, there can be > 1 light edge crossing it (*When?*)



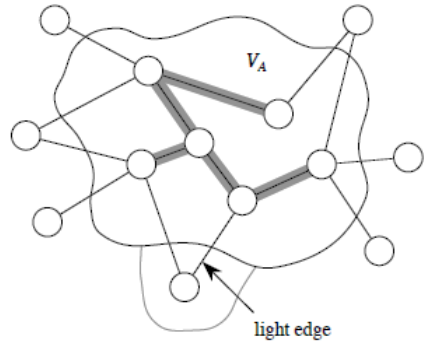
Minimum Spanning Tree

- **Min-weight Edge Theorem:**
  - Let T be MST of G, and let $A \subset T$ be a subtree of T
  - Let (u, v) be min-weight edge connecting A to $V - A$
  - Then, $(u, v) \in T$
- **Proof by contradiction**
  - Suppose T does not contain (u, v)
  - Then, some other edge (x, y) in T crosses the two sets
  - Create T' by replacing (x, y) with (u, v)
  - $w(T') < w(T)$ since $w(u, v) < w(x, y)$
  - => Contradiction



Prim's Algorithm

- Starts from an arbitrary "root" r and grows a MST A
- At each step, finds a light edge crossing cut $(V_A, V - V_A)$, where V_A = vertices that A is **incident** (**πρόσκειται**) on. Add this edge to A.

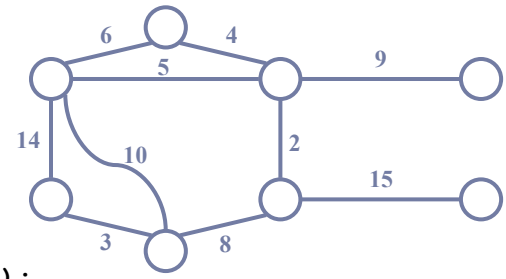


Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```


Prim's Algorithm

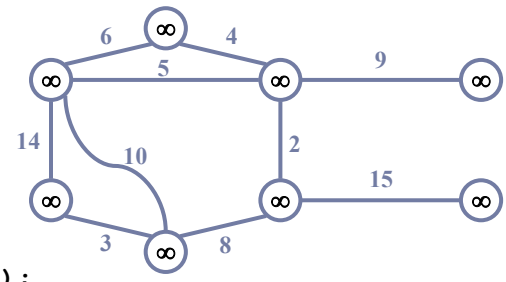
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Run on example graph

Prim's Algorithm

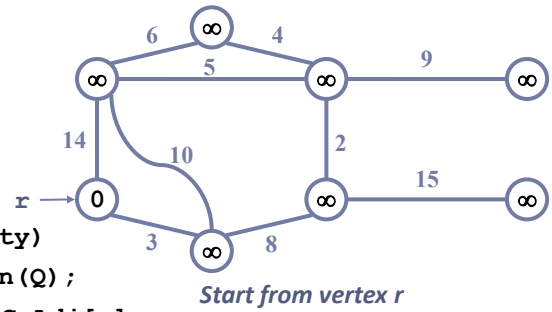
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Run on example graph

Prim's Algorithm

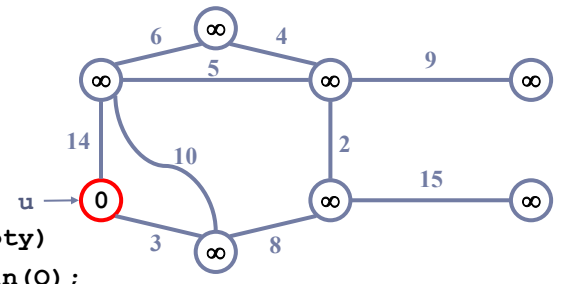
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Start from vertex r

Prim's Algorithm

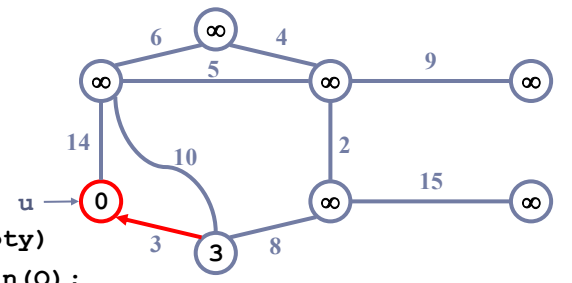
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Red vertices have been removed from Q

Prim's Algorithm

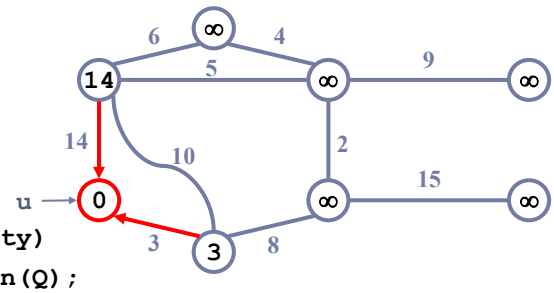
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Red arrows indicate parent pointers

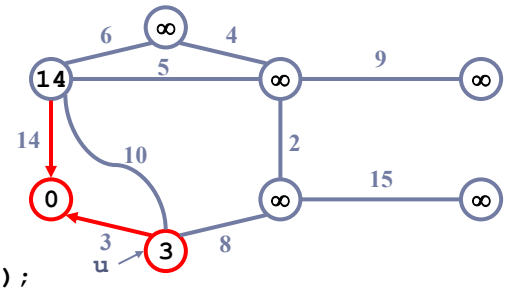
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



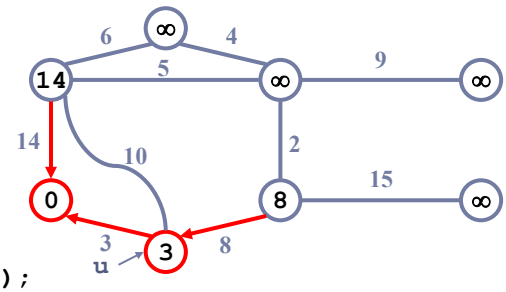
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



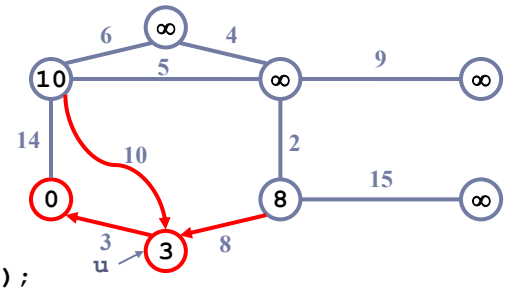
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



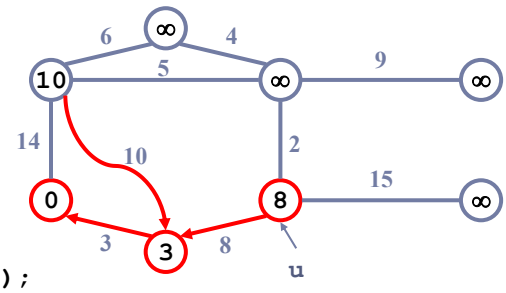
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



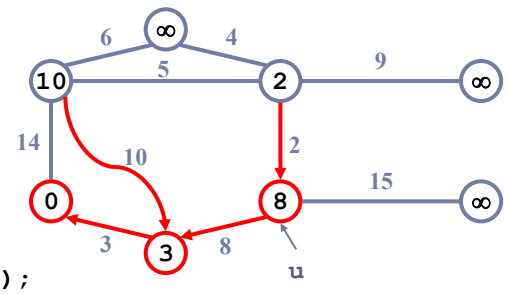
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



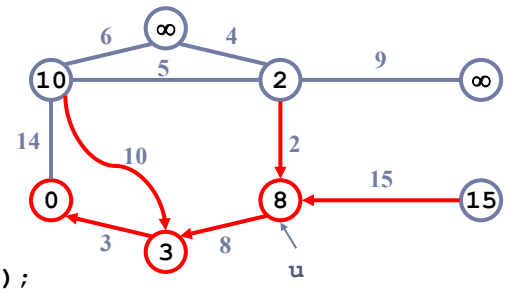
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



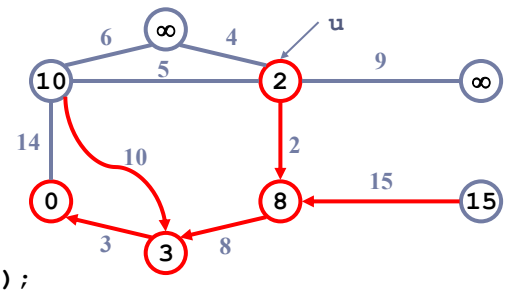
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



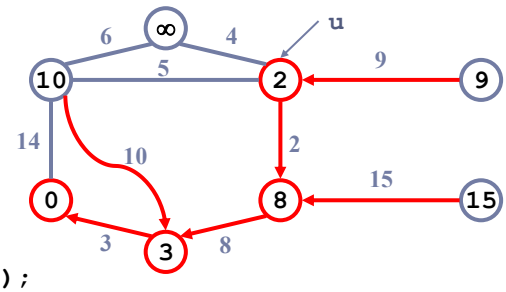
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Prim's Algorithm

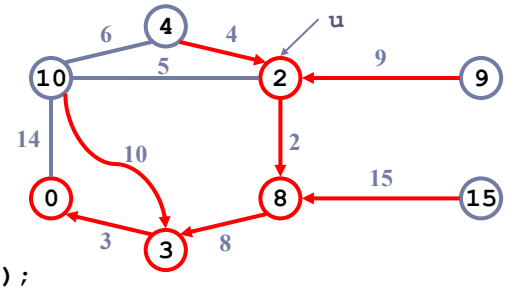
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



25

Prim's Algorithm

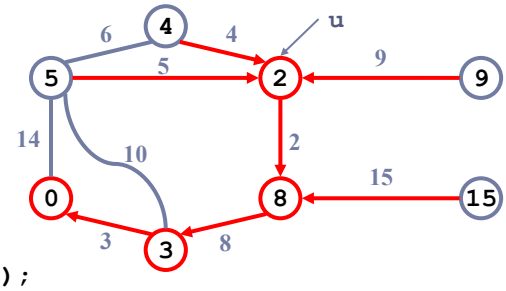
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



26

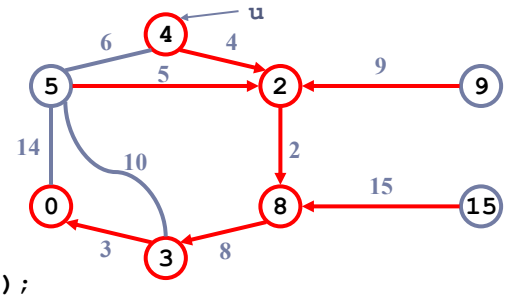
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



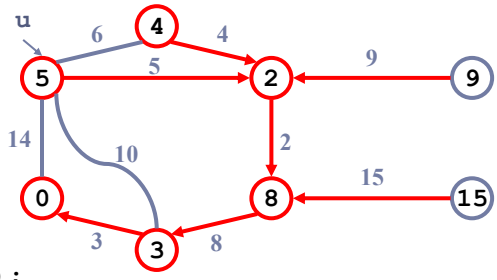
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



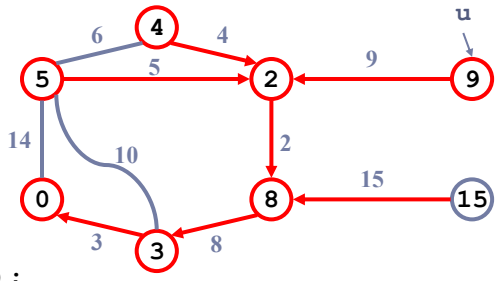
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



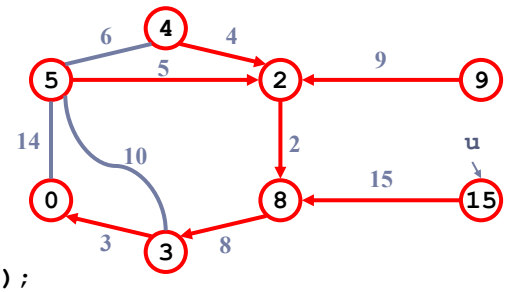
Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```



Review: Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            v.key = w(u,v);
```

What is the hidden cost in this code?

Review: Prim's Algorithm

```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            DecreaseKey(Q, v, w(u,v));
```

What is the hidden cost in this code?

Review: Prim's Algorithm

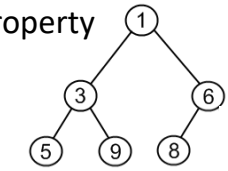
```
MST-Prim(G, w, r)
   Q = G.V;
   for each u ∈ Q
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            DecreaseKey(Q, v, w(u,v));
```

How often is ExtractMin() called? $|V|$

How often is DecreaseKey() called? $\leq |E|$

Heaps

- A **min-heap (σωρός)** is a tree-like data structure in which the values in the nodes satisfy the heap property
  $A[parent(i)] \leq A[i]$



- See CLRS chapter 6 for more information

Procedure	Binary heap (worst-case)	Binomial heap (worst-case)	Fibonacci heap (amortized)
MAKE-HEAP	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
INSERT	$\Theta(\lg n)$	$O(\lg n)$	$\Theta(1)$
MINIMUM	$\Theta(1)$	$O(\lg n)$	$\Theta(1)$
EXTRACT-MIN	$\Theta(\lg n)$	$\Theta(\lg n)$	$O(\lg n)$
UNION	$\Theta(n)$	$O(\lg n)$	$\Theta(1)$
DECREASE-KEY	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
DELETE	$\Theta(\lg n)$	$\Theta(\lg n)$	$O(\lg n)$

Review: Prim's Algorithm

Running time using Binary Heap

```
MST-Prim(G, w, r)
   Q = G.V;                          ← O(V)
   for each u ∈ Q                    ← O(V)
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);             ← O(V lg V)
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            DecreaseKey(Q, v, w(u,v)); ← O(E lg V)
```

Total: $O(E \lg V)$

Review: Prim's Algorithm

Running time using Fibonacci Heap

```
MST-Prim(G, w, r)
   Q = G.V;                          ← O(V)
   for each u ∈ Q                    ← O(V)
      u.key = ∞;
      u.p = NULL;
   r.key = 0;
   while (Q not empty)
      u = ExtractMin(Q);             ← O(V lg V)
      for each v ∈ G.Adj[u]
         if (v ∈ Q and w(u,v) < v.key)
            v.p = u;
            DecreaseKey(Q, v, w(u,v)); ← O(E)
```

Total: $O(E + V \lg V)$